



React

+



Vite

+



TypeScript

Índice

1. Creación de una nueva rama
2. Conexión del login con la base de datos
3. Creación del menú común
4. Evitar navegar a /home sin estar logueado (useEffect)
5. Creación del componente Dashboard
6. Creación del formulario en Dashboard
7. Creación del archivo backend/services/items.js
8. Funcionalidad del botón Insertar
9. Endpoints /getItem y /deleteItem
10. Creación de la tabla y lógica para mostrar/borrar datos
11. Ejecución del programa
12. Enlace al GitHub

1. Creación de una nueva rama

Crea una nueva rama en tu proyecto: te sitúas en la carpeta `proyectokat` y ahí escribes el siguiente comando:

git checkout -b crud

Con esto estás creando una rama nueva llamada `crud` y además, te estás cambiando a esa rama. Comprueba que estás en la nueva rama con el comando:

git branch

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● karen_at@karensilla-2 proyectosReactUT2 % cd proyectokat
● karen_at@karensilla-2 proyectokat % git checkout -b crud
Switched to a new branch 'crud'
● karen_at@karensilla-2 proyectokat % git branch
* crud
  main
  redux
○ karen_at@karensilla-2 proyectokat %
```

2. Conexión del login con la base de datos

Con lo que se explicó en clase sobre el backend y los endpoints, reemplaza tu comprobación de usuario y login correcto con la llamada al endpoint `/login` para conectarte a la base de datos y hacer la autenticación del usuario.

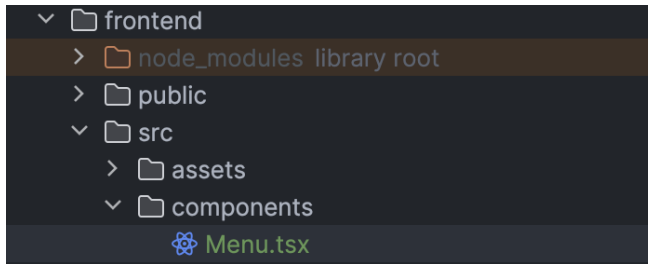
```
proyectokat > frontend > src > pages > Login.tsx > Login > handleInfo

1  import { Container, Typography, Button, Stack, TextField } from "@mui/material";
2  import { useState } from "react";
3  import { useDispatch } from "react-redux";
4  import { authActions } from "../store/authSlice";
5  import { useNavigate } from "react-router-dom";
6
7  export default function Login() {
8      const dispatch = useDispatch();
9      const navigate = useNavigate();
10
11      const [user, setUser] = useState('');
12      const [password, setPassword] = useState('');
13      const [loading] = useState(false);
14
15      const handleLogin = async () => {
16          if (!user || !password) {
17              alert('Por favor, completa usuario y contraseña');
18              return;
19          }
20
21          try {
22              const response = await fetch('http://localhost:3030/login?user=${user}&password=${password}');
23              const result = await response.json();
24
25              console.log('Respuesta del backend:', result);
26
27              if (result.data && result.data.length > 0) {
28                  const userData = result.data[0];
29
30                  dispatch(authActions.login({
31                      userName: userData.nombre,
32                      userRol: userData.rol,
33                      isAuthenticated: true
34                  }));
35
36                  alert('Login correcto');
37                  navigate('/home');
38              } else {
39                  alert('Usuario y/o contraseña incorrectos');
40              }
41          } catch (error) {
42              console.error('Error en login:', error);
43              alert('Error de conexión con el servidor');
44          }
45      };
46
47      const handleInfo = () => {
48          alert('Usuarios en la base de datos:\n--> admin / admin123\n--> user / user123\n--> invitado / invitado');
49      };
50
51      const handleCancel = () => {
52          setUser('');
53          setPassword('');
54      };
55  }
```

3. Creación del menú común a todas las páginas

El menú será común a todas las páginas de la aplicación: tanto si estamos en Inicio (en `/home`) como si estamos en Informes (`/reports`) queremos ver el mismo menú. Por lo tanto lo vamos a programar en un fichero que

estará dentro de la carpeta components: en el **frontend/src/components/Menu.tsx**.



Tendrás que trasladar la parte de la navegación entre páginas (`useNavigate`, `useDispatch`, `useSelector`) a esa parte del código, puesto que será común en todas las páginas.

```

projectoktat > frontend > src > components > Menu.tsx > Menu > toggleDrawer
26
27 function Menu() {
28   //Almacenamos en la variable userData lo que obtenemos del store usando el hook useSelector
29   const userData = useSelector((state: RootState) => state.authenticator)
30   const navigate = useNavigate();
31   const dispatch = useDispatch();
32
33   //Evitar que se pueda acceder a paginas sin estar autenticado
34   const isLoggedIn = userData.isAuthenticated
35   useEffect(() => {
36     if (!isLoggedIn) {
37       navigate('/')
38     }
39   }, [isLoggedIn, navigate])
40
41   //Comprobamos por la consola qué obtenemos del store
42   console.log(userData)
43
44   //Drawer
45   const [open, setOpen] = React.useState(false);
46   const toggleDrawer = (newOpen: boolean) => () => {
47     setOpen(newOpen);
48   };
49
50   //cerrar sesion
51   function handleLogout() {
52     dispatch(authActions.logout())
53     navigate('/')
54   }
55
56   const DrawerList = (
57     <Box sx={{width: 250}} role="presentation" onClick={toggleDrawer(false)}>
58       {/*Link a las distintas paginas de la app*/}
59       <List>
60         <Link to={'/home'}>
61           <ListItemButton>
62             <ListItemIcon sx={{color: 'secondary.main'}}>
63               <CottageIcon/>
64             </ListItemIcon>
65             <ListItemText sx={{color: 'primary.main'}}>Inicio</ListItemText>
66           </ListItemButton>
67         </Link>
68
69         {/*Solo el usuario admin puede ver la seccion reports*/}
70
71         <Link to={'/reports'}>
72           <ListItemButton>
73             <ListItemIcon sx={{color: 'secondary.main'}}>
74               <FeedIcon/>
75             </ListItemIcon>
76             <ListItemText sx={{color: 'primary.main'}}>Informes</ListItemText>
77           </ListItemButton>
78         </Link>
79

```

4. Cómo evitar navegar a /home sin estar logueado: hook useEffect()

Para implementar esta protección en el menú, modificamos el *Menu.tsx* añadiendo el hook *useEffect*:

```
const DrawerList = (<Box sx={{width: 250}} role="presentation" onClick={toggleDrawer(false)}>
  { /*Link a las distintas paginas de la app*/ }
  <List>
    <Link to={'/home'}>
      <ListItemButton>
        <ListItemIcon sx={{color: 'secondary.main'}}>
          <CottageIcon/>
        </ListItemIcon>
        <ListItemText sx={{color: 'primary.main'}}>Inicio</ListItemText>
      </ListItemButton>
    </Link>

    { /*Solo el usuario admin puede ver la seccion reports*/ }

    <Link to={'/reports'}>
      <ListItemButton>
        <ListItemIcon sx={{color: 'secondary.main'}}>
          <FeedIcon/>
        </ListItemIcon>
        <ListItemText sx={{color: 'primary.main'}}>Informes</ListItemText>
      </ListItemButton>
    </Link>

    { /*Solo el usuario admin puede gestionar usuarios*/ }
    { userData.userRol === "admin" && }
    <Link to={'/gestionusuarios'}>
      <ListItemButton>
        <ListItemIcon sx={{color: 'secondary.main'}}>
          <ManageAccountsIcon/>
        </ListItemIcon>
        <ListItemText sx={{color: 'primary.main'}}>Gestionar Usuarios</ListItemText>
      </ListItemButton>
    </Link>

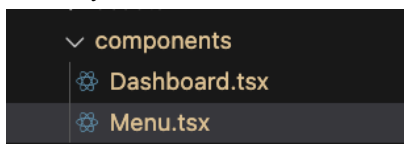
    <Link to={'/help'}>
      <ListItemButton>
        <ListItemIcon sx={{color: 'secondary.main'}}>
          <HelpIcon/>
        </ListItemIcon>
        <ListItemText sx={{color: 'primary.main'}}>Ayuda</ListItemText>
      </ListItemButton>
    </Link>

    <ListItemButton onClick={handleLogout}>
      <ListItemIcon sx={{color: 'secondary.main'}}>
        <OutputIcon/>
      </ListItemIcon>
      <ListItemText sx={{color: 'primary.main'}}>Salir</ListItemText>
    </ListItemButton>
  </List>
```

Ahora cuando ponemos <http://localhost:5173/home> o <http://localhost:5173/reports> directamente en la URL, no nos deja entrar puesto que no estamos logueados, y nos redirige a la página de *login*.

5. Creación del componente <Dashboard> que usarás en Home.tsx

Crea un componente llamado Dashboard en el fichero frontend/src/components/Dahsboard.tsx. Este componente tendrá el formulario y la tabla que se usarán en la página /home. Es decir, ahora el fichero Home.tsx renderizará los componentes <Menu> y <Dashboard> en su return



```

proyectokat > frontend > src > components > Dashboard.tsx > ...
1  import { useEffect, useState } from "react";
2  import { Box, Button, Container, TextField, Divider, Table, TableContainer, TableHead, TableRow, TableCell, TableBody } from "@mui/material";
3  import SendIcon from '@mui/icons-material/Send';
4  import DeleteIcon from '@mui/icons-material/Delete';
5  import DeleteForeverIcon from '@mui/icons-material/DeleteForever';
6  import { useSelector } from "react-redux";
7  import type { RootState } from "../store/index";
8
9  // Creamos el tipo itemType fuera del componente
10 interface ItemType {
11   id: number;
12   nombre: string;
13   marca: string;
14   tipo: string;
15   precio: number;
16 }
17
18 // Inicializo los valores del item
19 const itemInitialState: ItemType = {
20   id: 0,
21   nombre: '',
22   marca: '',
23   tipo: '',
24   precio: 0,
25 };
26
27 export default function Dashboard() {
28   // Estado para los datos de la tabla
29   const [tableData, setTableData] = useState<ItemType[]>([]);
30
31   // Estado para el formulario
32   const [item, setItem] = useState<ItemType>(itemInitialState);
33
34   // Almacenamos en la variable userData lo que obtenemos del store
35   const userData = useSelector((state: RootState) => state.authenticator);
36
37   // Cargar datos al montar el componente
38   useEffect(() => {
39     getItems();
40   }, []);
41
42   // Manejar cambios en los inputs de forma más sencilla
43   const handleInputChange = (field: keyof ItemType) => (e: React.ChangeEvent<HTMLInputElement>) => {
44     let value;
45
46     if (field === 'precio') {
47       const parsedValue = parseFloat(e.target.value);
48       if (isNaN(parsedValue)) {
49         value = 0;
50       } else {
51         value = parsedValue;
52       }
53     } else {
54       value = e.target.value;
55     }
56   };
57
58   // Función para obtener los items
59   const getItems = () => {
60     // Aquí iría la lógica para obtener los items desde el backend o una API
61     // Por ejemplo: fetch('/api/items').then(response => response.json());
62   };
63
64   // Función para enviar el formulario
65   const handleSubmit = () => {
66     // Aquí iría la lógica para enviar el formulario al backend o una API
67     // Por ejemplo: fetch('/api/items', { method: 'POST', body: JSON.stringify(item) });
68   };
69
70   // Función para eliminar un item
71   const deleteItem = (id: number) => {
72     // Aquí iría la lógica para eliminar un item desde el backend o una API
73     // Por ejemplo: fetch(`/api/items/${id}`, { method: 'DELETE' });
74   };
75
76   // Función para eliminar un item de forma permanente
77   const deleteForeverItem = (id: number) => {
78     // Aquí iría la lógica para eliminar un item de forma permanente desde el backend o una API
79     // Por ejemplo: fetch(`/api/items/${id}`, { method: 'DELETE', headers: { 'X-Delete-Forever': 'true' } });
80   };
81
82   // Renderizado de la tabla
83   const renderTable = () => {
84     return (
85       <Table>
86         <TableHead>
87           <TableRow>
88             <TableCell>ID</TableCell>
89             <TableCell>Nombre</TableCell>
90             <TableCell>Marca</TableCell>
91             <TableCell>Tipo</TableCell>
92             <TableCell>Precio</TableCell>
93             <TableCell>Acciones</TableCell>
94           </TableRow>
95         </TableHead>
96         <TableBody>
97           {tableData.map((item) => (
98             <TableRow>
99               <TableCell>{item.id}</TableCell>
100              <TableCell>{item.nombre}</TableCell>
101              <TableCell>{item.marca}</TableCell>
102              <TableCell>{item.tipo}</TableCell>
103              <TableCell>{item.precio}</TableCell>
104              <TableCell>
105                <Button>Eliminar</Button>
106                <Button>Eliminar Permanente</Button>
107              </TableCell>
108            </TableRow>
109          ))}
110        </TableBody>
111      </Table>
112    );
113  };
114
115   // Renderizado del formulario
116   const renderForm = () => {
117     return (
118       <Form>
119         <FormLabel>Agregar nuevo item</FormLabel>
120         <FormText>
121           <FormText>Nombre</FormText>
122           <FormText>Marca</FormText>
123           <FormText>Tipo</FormText>
124           <FormText>Precio</FormText>
125         </FormText>
126         <FormButton>Enviar</FormButton>
127       </Form>
128     );
129   };
130
131   // Renderizado de la interfaz
132   return (
133     <Container>
134       <Header>Dashboard</Header>
135       <TableContainer>
136         <Table>
137           <TableHead>
138             <TableRow>
139               <TableCell>ID</TableCell>
140               <TableCell>Nombre</TableCell>
141               <TableCell>Marca</TableCell>
142               <TableCell>Tipo</TableCell>
143               <TableCell>Precio</TableCell>
144               <TableCell>Acciones</TableCell>
145             </TableRow>
146           </TableHead>
147           <TableBody>
148             {tableData.map((item) => (
149               <TableRow>
150                 <TableCell>{item.id}</TableCell>
151                 <TableCell>{item.nombre}</TableCell>
152                 <TableCell>{item.marca}</TableCell>
153                 <TableCell>{item.tipo}</TableCell>
154                 <TableCell>{item.precio}</TableCell>
155                 <TableCell>
156                   <Button>Eliminar</Button>
157                   <Button>Eliminar Permanente</Button>
158                 </TableCell>
159               </TableRow>
160             ))}
161           </TableBody>
162         </Table>
163       </TableContainer>
164       <Form>
165         <FormLabel>Agregar nuevo item</FormLabel>
166         <FormText>
167           <FormText>Nombre</FormText>
168           <FormText>Marca</FormText>
169           <FormText>Tipo</FormText>
170           <FormText>Precio</FormText>
171         </FormText>
172         <FormButton>Enviar</FormButton>
173       </Form>
174     </Container>
175   );
176 }

```

Además modificamos el Home.tsx:

```

projectokai > frontend > src > pages > Home.tsx > Home
1  import { Container, Typography, Stack, Button, Box } from "@mui/material";
2  import { useSelector, useDispatch } from "react-redux";
3  import type { RootState } from "../store/index";
4  import { authActions } from "../store/authSlice";
5  import { useNavigate } from "react-router-dom";
6  import Menu from "../components/Menu";
7  import Dashboard from "../components/Dashboard";
8
9  export default function Home() {
10     const userData = useSelector((state: RootState) => state.authenticator);
11     const dispatch = useDispatch();
12     const navigate = useNavigate();
13
14     console.log("Datos del usuario desde el store:", userData);
15
16     const handleLogout = () => {
17         dispatch(authActions.logout());
18         navigate("/");
19     };
20
21     return (
22         <Menu />
23         <Container maxWidth="lg" sx={{ mt: 4 }}>
24             { /* Tu contenido actual */ }
25             <Box sx={{ mb: 4 }}>
26                 <Stack spacing={3} alignItems="center">
27                     <Typography variant="h1" color="primary">
28                         Bienvenido/a
29                     </Typography>
30
31                     <Typography variant="body1" align="center" sx={{ maxWidth: 350 }}>
32                         Home de Karen: Soy el usuario <strong>{userData.userName}</strong> y tengo el ro
33                     </Typography>
34
35                     <Button
36                         variant="contained"
37                         color="primary"
38                         onClick={handleLogout}
39                     >
40                         Salir
41                     </Button>
42                 </Stack>
43             </Box>
44
45             { /* Dashboard con formulario y tabla */ }
46             <Dashboard />
47         </Container>
48     );
49 }
50
51

```

6. Creación del formulario en Dashboard.tsx

```
pyectokat > frontend > src > components > Dashboard.tsx > ...
1 import { useEffect, useState } from "react";
2 import {Box, Button, Container, TextField, Divider, Table, TableContainer, TableHead, TableRow, TableCell, Tab
3 import SendIcon from '@mui/icons-material/Send';
4 import DeleteIcon from '@mui/icons-material/Delete';
5 import DeleteForeverIcon from '@mui/icons-material/DeleteForever';
6 import { useSelector } from "react-redux";
7 import type { RootState } from "../store/index";
8
9 // Creamos el tipo itemType fuera del componente
10 interface ItemType {
11   id: number;
12   nombre: string;
13   marca: string;
14   tipo: string;
15   precio: number;
16 }
17
18 // Inicializo los valores del item
19 const itemInitialState: ItemType = {
20   id: 0,
21   nombre: '',
22   marca: '',
23   tipo: '',
24   precio: 0,
25 };
26
27 export default function Dashboard() {
28   // Estado para los datos de la tabla
29   const [tableData, setTableData] = useState<ItemType[]>([]);
30
31   // Estado para el formulario
32   const [item, setItem] = useState<ItemType>({itemInitialState});
33
34   // Almacenamos en la variable userData lo que obtenemos del store
35   const userData = useSelector((state: RootState) => state.authenticator);
36
37   // Cargar datos al montar el componente
38   useEffect(() => {
39     getItems();
40   }, []);
41
42   // Manejar cambios en los inputs de forma más sencilla
43   const handleInputChange = (field: keyof ItemType) => (e: React.ChangeEvent<HTMLInputElement>) => {
44     let value;
45
46     if (field === 'precio') {
47       const parsedValue = parseFloat(e.target.value);
48       if (isNaN(parsedValue)) {
49         value = 0;
50       } else {
51         value = parsedValue;
52       }
53     } else {
54       value = e.target.value;
55     }
56   }
57 }
```

7. Creación del fichero backend/services/items.js


```

proyectokat > backend > services > JS items.js > ...
1  import db from './db.js';
2  import helper from './helper.js';
3
4  async function insertData(req, res) {
5      const data = req.query;
6      await db.query(
7          `INSERT INTO coleccion (nombre, marca, tipo, precio) VALUES (:nombre, :marca, :tipo, :precio)`,
8          {
9              nombre: data.nombre,
10             marca: data.marca,
11             tipo: data.tipo,
12             precio: data.precio
13         }
14     );
15     return 1; // Simulamos affectedRows
16 }
17
18 async function getData(req, res) {
19     const rows = await db.query(`SELECT * FROM coleccion`);
20     const data = helper.emptyOrRows(rows);
21     return {
22         data
23     };
24 }
25
26 async function deleteData(req, res) {
27     const data = req.query;
28     await db.query(
29         `DELETE FROM coleccion WHERE id = :id`,
30         { id: data.id }
31     );
32     return 1; // Simulamos affectedRows
33 }
34
35 export default {
36     getData,
37     insertData,
38     deleteData
39 };

```

8. Añadir funcionalidad al botón Insertar del formulario.

```

dex.ts  Login.tsx .../frontend/... M  Menu.tsx .../frontend/... M  Dashboard.tsx M  JS items.js M X  Home.ts
proyectokat > backend > services > JS items.js > ...
1  import db from './db.js';
2  import helper from './helper.js';
3
4  async function insertData(req, res) {
5      const data = req.query;
6      await db.query(
7          `INSERT INTO coleccion (nombre, marca, tipo, precio) VALUES (:nombre, :marca, :tipo, :precio)`,
8          {
9              nombre: data.nombre,
10             marca: data.marca,
11             tipo: data.tipo,
12             precio: data.precio
13         }
14     );
15     return 1; // Simulamos affectedRows
16 }
17
18 async function getData(req, res) {
19     const rows = await db.query(`SELECT * FROM coleccion`);
20     const data = helper.emptyOrRows(rows);
21     return {
22         data
23     };
24 }
25
26 async function deleteData(req, res) {
27     const data = req.query;
28     await db.query(
29         `DELETE FROM coleccion WHERE id = :id`,
30         { id: data.id }
31     );
32     return 1; // Simulamos affectedRows
33 }
34
35 export default {
36     getData,
37     insertData,
38     deleteData
39 };

```

9. Creación del resto de los endpoints: /getItem y /deleteItem

9.1 Obtener datos de la base de datos (/getItems)

Para probar que funciona vamos al *endpoint* /getItems a través del navegador (recuerden que los endpoints están en el puerto 3030): <http://localhost:3030/getItems>



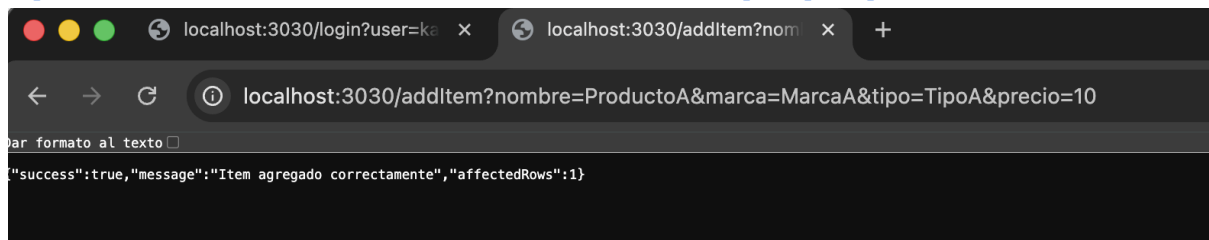
```
localhost:3030/getItems
localhost:3030/getItems

Dar formato al texto
{"data":[{"id":4,"nombre":"ad","marca":"ad","tipo":"ada","precio":2.00}, {"id":5,"nombre":"sonia","marca":"sc","tipo":"sc","precio":12.00}, {"id":6,"nombre":"dw","marca":"wdw","tipo":"dw","precio":23.00}, {"id":7,"nombre":"acd","marca":"ac","tipo":"ac","precio":1.00}, {"id":8,"nombre":"adc","marca":"ac","tipo":"ac","precio":4.00}, {"id":9,"nombre":"svd","marca":"sv","tipo":"sv","precio":1.00}, {"id":10,"nombre":"wdcsw","marca":"ac","tipo":"cd","precio":50.00}, {"id":11,"nombre":"adv","marca":"dsva","tipo":"adv","precio":40.00}, {"id":12,"nombre":"sonia","marca":"ac","tipo":"ac","precio":1.00}, {"id":13,"nombre":"dcs","marca":"dcs","tipo":"dcs","precio":1.00}, {"id":14,"nombre":"vh","marca":"dacia","tipo":"puta","precio":1.00}]}
```

9.2 Borrar un registro de la base de datos (/deleteItem)

Primero probamos insertando uno de los datos

<http://localhost:3030/addItem?nombre=ProductoA&marca=MarcaA&tipo=TipoA&precio=10>

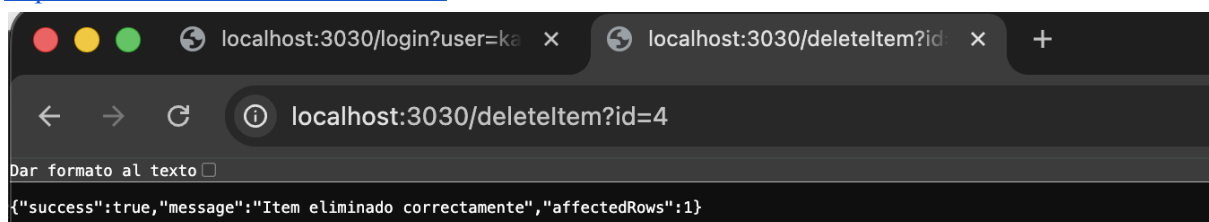


```
localhost:3030/login?user=ka localhost:3030/addItem?nom localhost:3030/addItem?nom
localhost:3030/addItem?nombre=ProductoA&marca=MarcaA&tipo=TipoA&precio=10

Dar formato al texto
{"success":true,"message":"Item agregado correctamente","affectedRows":1}
```

Vamos a probar borrando el id 4

<http://localhost:3030/deleteItem?id=4>

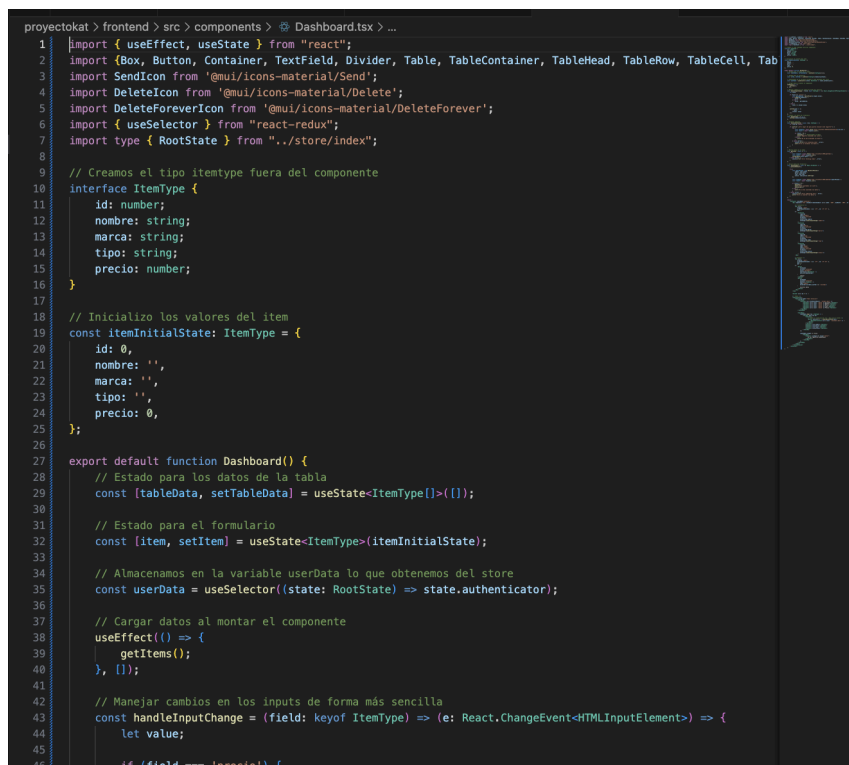


```
localhost:3030/login?user=ka localhost:3030/deleteItem?id=4
localhost:3030/deleteItem?id=4

Dar formato al texto
{"success":true,"message":"Item eliminado correctamente","affectedRows":1}
```

10. Creación de la tabla en el fichero Dashboard.tsx: lógica para mostrar los datos en la tabla y lógica para borrar un registro de la tabla

10.1 Lógica para mostrar los datos en la tabla



```
proyectokat > frontend > src > components > Dashboard.tsx > ...
1 import { useEffect, useState } from "react";
2 import { Box, Button, Container, TextField, Divider, Table, TableContainer, TableHead, TableRow, TableCell, Tab
3 import SendIcon from '@mui/icons-material/Send';
4 import DeleteIcon from '@mui/icons-material/Delete';
5 import DeleteForeverIcon from '@mui/icons-material/DeleteForever';
6 import { useSelector } from "react-redux";
7 import type { RootState } from "../store/index";
8
9 // Creamos el tipo ItemType fuera del componente
10 interface ItemType {
11   id: number;
12   nombre: string;
13   marca: string;
14   tipo: string;
15   precio: number;
16 }
17
18 // Inicializo los valores del item
19 const itemInitialState: ItemType = {
20   id: 0,
21   nombre: '',
22   marca: '',
23   tipo: '',
24   precio: 0,
25 };
26
27 export default function Dashboard() {
28   // Estado para los datos de la tabla
29   const [tableData, setTableData] = useState<ItemType[]>([]);
30
31   // Estado para el formulario
32   const [item, setItem] = useState<ItemType>(itemInitialState);
33
34   // Almacenamos en la variable userData lo que obtenemos del store
35   const userData = useSelector((state: RootState) => state.authenticator);
36
37   // Cargar datos al montar el componente
38   useEffect(() => {
39     getItems();
40   }, []);
41
42   // Manejar cambios en los inputs de forma más sencilla
43   const handleInputChange = (field: keyof ItemType) => (e: React.ChangeEvent<HTMLInputElement>) => {
44     let value;
45
46     if (field === 'precio') {
```

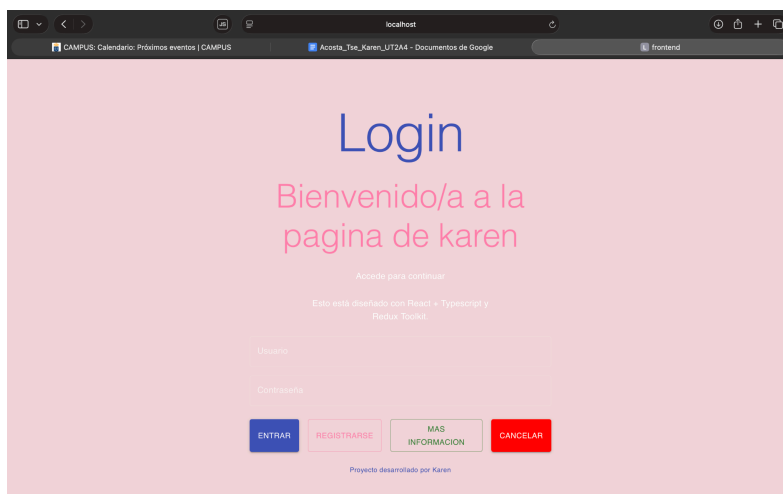
10.2 Implementación de la tabla: componente Table

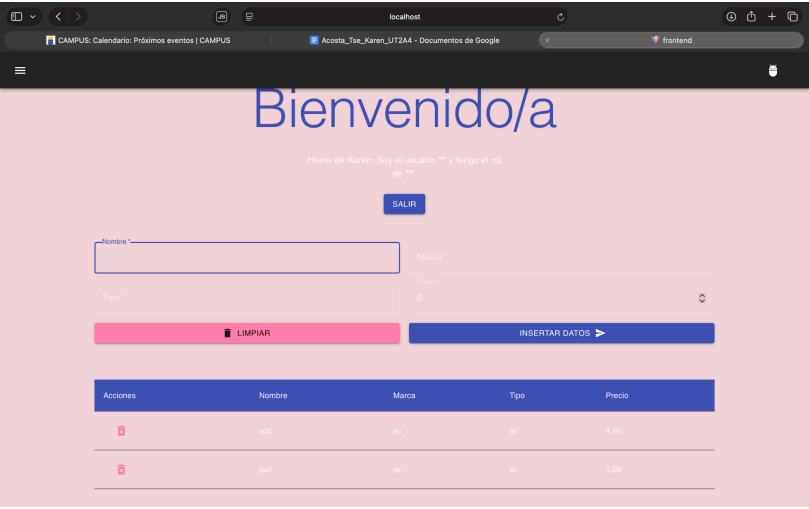
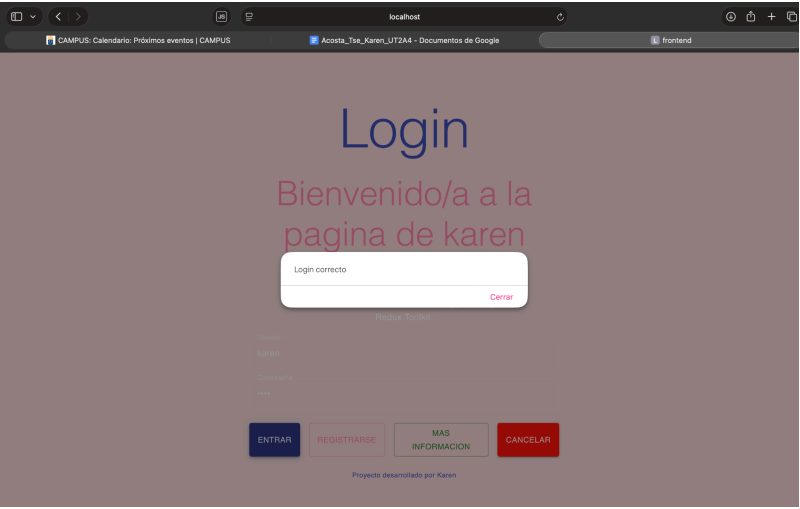
```
    { /* Tabla */  
    <TableContainer>  
      <Table aria-label='Tabla Coleccion'>  
        <TableHead>  
          <TableRow sx={{ backgroundColor: 'primary.main' }}>  
            <TableCell sx={{ color: 'white' }}>Acciones</TableCell>  
            <TableCell sx={{ color: 'white' }}>Nombre</TableCell>  
            <TableCell sx={{ color: 'white' }}>Marca</TableCell>  
            <TableCell sx={{ color: 'white' }}>Tipo</TableCell>  
            <TableCell sx={{ color: 'white' }}>Precio</TableCell>  
          </TableRow>  
        </TableHead>  
  
        <TableBody>  
          {tableData.map((row: ItemType) => (  
            <TableRow key={row.id}>  
              <TableCell>  
                { /* QUITA la condición del rol - deja solo el botón */  
                <Button onClick={() => handleDeleteItem(row)}>  
                  <DeleteForeverIcon sx={{ color: 'secondary.main' }} />  
                </Button>  
              </TableCell>  
              <TableCell>{row.nombre}</TableCell>  
              <TableCell>{row.marca}</TableCell>  
              <TableCell>{row.tipo}</TableCell>  
              <TableCell>{row.precio}</TableCell>  
            </TableRow>  
          ))}  
  
          {tableData.length === 0 && (  
            <TableRow>  
              <TableCell colspan={5} align="center">  
                No hay registros disponibles  
              </TableCell>  
            </TableRow>  
          )}  
        </TableBody>  
      </Table>  
    </TableContainer>  
  </Container>  
);  
}
```

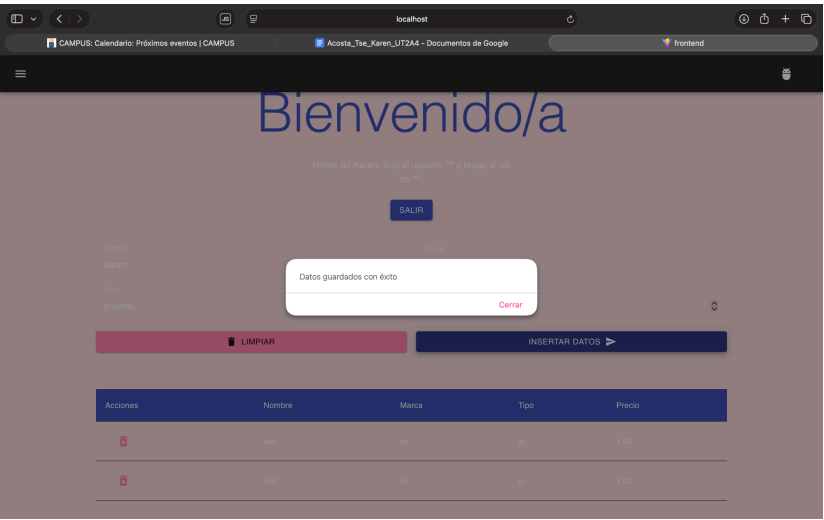
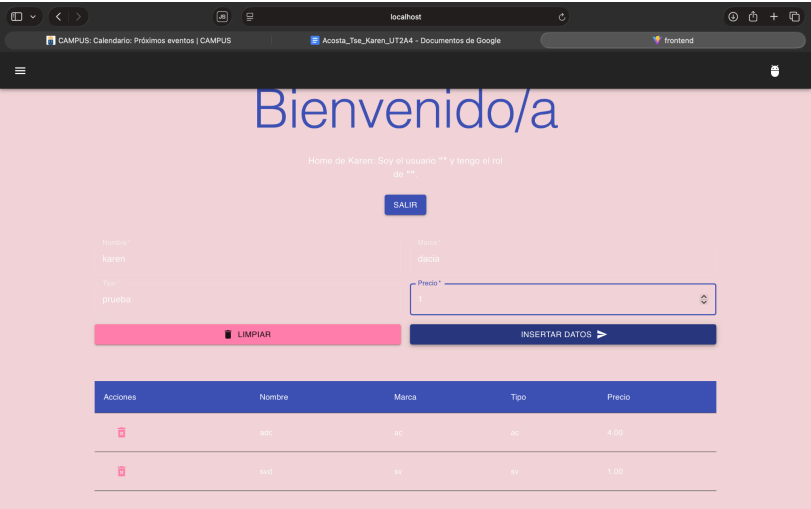
10.3 Lógica para borrar un registro de la tabla

```
proyektokat > frontend > src > components > Dashboard.tsx > ...  
1 import { useEffect, useState } from "react";  
2 import { Box, Button, Container, TextField, Divider, Table, TableContainer, TableHead, TableRow, TableCell, Tab  
3 import SendIcon from '@mui/icons-material/Send';  
4 import DeleteIcon from '@mui/icons-material/Delete';  
5 import DeleteForeverIcon from '@mui/icons-material/DeleteForever';  
6 import { useSelector } from "react-redux";  
7 import type { RootState } from "../store/index";  
8
```

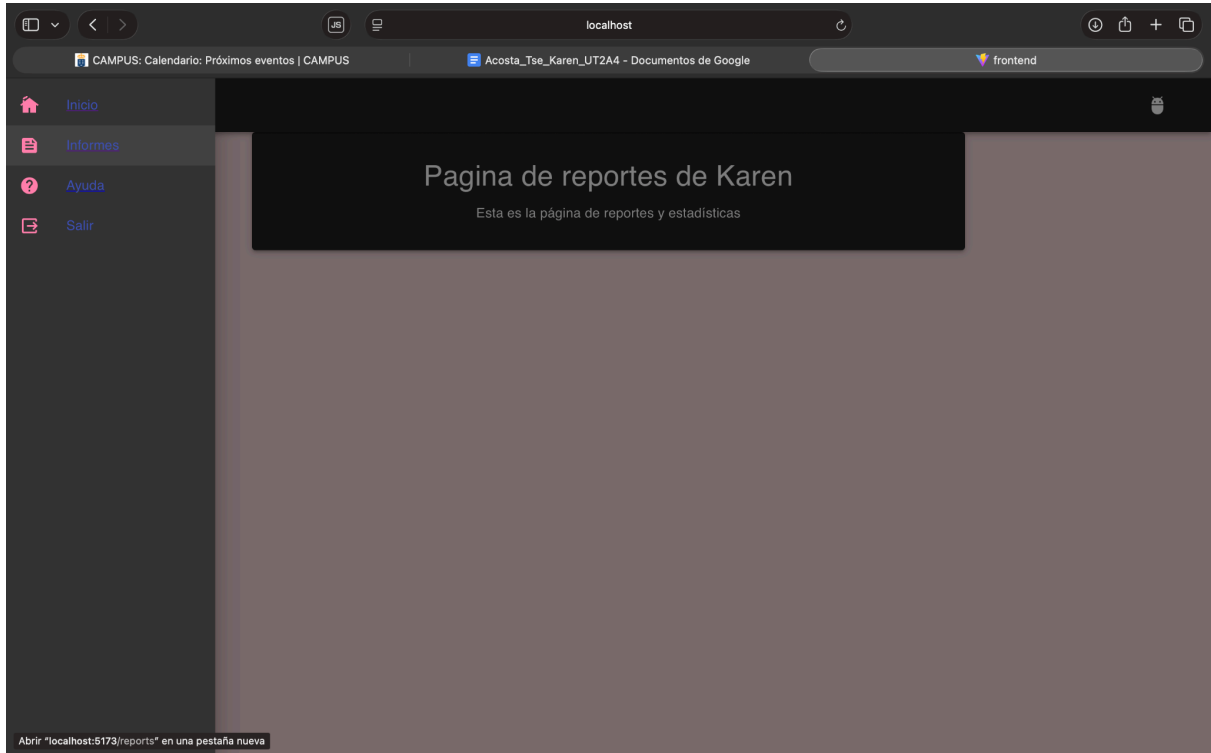
11. Ejecución del programa







Si le damos a informes nos lleva a la página de reportes:



12. Enlace al github

<https://github.com/karenat2706/proyectokat.git>